Simple Expense Tracker – API Design

Oct 13, 2026 · @skr

1. Conventions

The API is a JSON REST API under `/v1`, used by the web app and the Android and iOS apps alike. Every endpoint below works with the tables in the Database Design doc.

Topic	Rule
Base URL	`https://api.<yourdomain>/v1`
Format	Requests and responses are `application/json`, UTF-8
Auth	`Authorization: Bearer <access_token>`; access token lives 15 minutes, refresh token 30 days
Money	Integer paise in fields ending `_paise` (₹10.50 = 1050); the app formats it as ₹
Dates	`spent_on` is `YYYY-MM-DD` in the user's time zone; timestamps are ISO 8601 UTC
IDs	UUIDs. For expenses, the client creates the ID, so retrying a create never makes a duplicate
Pagination	`?limit=50&cursor=<opaque>`; responses include `next_cursor` (null at the end). Max limit 100
Idempotency	`PUT /expenses/{id}` is idempotent. Other POSTs accept an `Idempotency-Key` header
Rate limits	100 requests per minute per user; OTP requests 5 per hour per phone. Exceeding returns 429 with `Retry-After`
Versioning	Breaking changes go to `/v2`; adding fields is not breaking, so clients must ignore unknown fields

Error format. All errors use the same shape:

```
{
  "error": {
    "code": "validation_error",
    "message": "amount_paise must be greater than 0",
    "field": "amount_paise"
  }
}
```

Guest mode. Guests do not call the API at all; the app stores data locally. The API is used only after sign-in (or for the optional sync).

2. Authentication

Users sign in with a phone OTP or Google. No passwords. Sign-in is optional; it only enables backup and sync.

Method	Path	Purpose
POST	`/auth/otp/request`	Send a 6-digit OTP to a phone number
POST	`/auth/otp/verify`	Verify the OTP and return tokens
POST	`/auth/google`	Sign in with a Google ID token
POST	`/auth/refresh`	Exchange a refresh token for new tokens
POST	`/auth/logout`	Revoke the current refresh token

Request an OTP

```
POST /v1/auth/otp/request
{ "phone": "+919876543210" }

200 OK
{ "expires_in": 300, "resend_after": 30 }
```

Verify the OTP and sign in

```
POST /v1/auth/otp/verify
{
  "phone": "+919876543210",
  "code": "482913",
  "device": { "platform": "web", "app_version": "1.0.0" }
}

200 OK
{
  "access_token": "eyJ...",
  "refresh_token": "rt_9f2c...",
  "expires_in": 900,
  "is_new_user": true,
  "user": {
    "id": "b1c0...", "phone": "+919876543210", "language": "en",
    "timezone": "Asia/Kolkata", "week_start": 1
  }
}
```

A wrong or expired code returns `401 invalid_otp`. After 5 wrong attempts the code is locked and the user must request a new one.

Google sign-in. `POST /auth/google` with `{ "id_token": "<Google ID token>", "device": {…} }` returns the same response as OTP verify.

Refresh. `POST /auth/refresh` with `{ "refresh_token": "rt_9f2c..." }` returns a new access token and a new refresh token. The old refresh token is revoked at once (rotation). If a revoked token is used again, all tokens of that device are revoked.

Merging guest data. After the first sign-in, the app calls `POST /sync/push` (section 6) with everything stored locally. No special merge endpoint is needed because the client-generated IDs make the upload safe.

3. Expenses, quick-add and categories

Method	Path	Purpose
PUT	`/expenses/{id}`	Create or update an expense (idempotent, ID chosen by the client)
GET	`/expenses`	List expenses with filters

Method	Path	Purpose
GET	`/expenses/{id}`	Get one expense
DELETE	`/expenses/{id}`	Soft delete
POST	`/expenses/{id}/restore`	Undo a delete (within 90 days)
GET	`/quick-add`	Get the user's chips, most used first
PUT	`/quick-add/{id}`	Create or update a chip; pin or unpin
DELETE	`/quick-add/{id}`	Remove a chip
GET	`/categories`	System and custom categories
POST	`/categories`	Create a custom category
GET	`/categories/suggest?q=samosa`	Guess a category and tag from an item name

Add an expense. If `category_id` is omitted, the server guesses it from the item name; if `tag` is omitted, the category's default tag is used. The server also updates the matching quick-add chip.

```
PUT /v1/expenses/7c9e6679-7425-40de-944b-e07fc1f90ae7
{
  "item_name": "Auto",
  "amount_paise": 1000,
  "spent_on": "2026-10-13",
  "tag": "need"
}

201 Created   (200 OK when it already existed and was updated)
{
  "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "item_name": "Auto",
  "amount_paise": 1000,
  "currency": "INR",
  "category": { "id": "3f1a...", "name": "Travel" },
  "tag": "need",
  "spent_on": "2026-10-13",
  "updated_at": "2026-10-13T09:12:44Z",
  "limit_status": [
```

```
    { "period": "week", "spent_paise": 41000, "limit_paise": 50000,
      "percent": 82, "new_alert_level": 80 }
  ]
}
```

The `limit_status` list lets the app show the "80% used" message straight after adding, without a second call. `new_alert_level` is present only when this expense crossed 80 or 100 for the first time in that period.

List expenses

```
GET /v1/expenses?from=2026-10-01&to=2026-10-
31&category_id=...&tag=want&q=tea&limit=50&cursor=...

200 OK
{
  "items": [ { "id": "...", "item_name": "Tea", "amount_paise": 1500, ... }
],
  "day_totals": { "2026-10-13": 6500, "2026-10-12": 12000 },
  "next_cursor": null
}
```

Edit uses the same `PUT` with the full object. **Delete** returns `204 No Content`; the app shows a 5-second Undo that calls `restore`.

Quick-add chips

```
GET /v1/quick-add
{
  "items": [
    { "id": "a1...", "item_name": "Auto", "default_amount_paise": 1000,
      "category_id": "3f1a...", "tag": "need", "is_pinned": true },
    { "id": "b2...", "item_name": "Tea", "default_amount_paise": 1500,
      "category_id": "5d2c...", "tag": "want", "is_pinned": false }
  ]
}
```

Suggest a category

```
GET /v1/categories/suggest?q=samosa
{ "category_id": "5d2c...", "category_name": "Food", "tag": "want",
"confidence": "keyword" }
```

When the user changes the suggested category, the app saves the expense with the new category, and the server stores that word as a learned keyword for this user (`category_keywords` with `user_id` set).

4. Limits

A user can have at most one limit for each period: week, month and year. All limits are optional.

Method	Path	Purpose
GET	`/limits`	All limits with current progress
PUT	`/limits/{period}`	Create or change a limit; `period` is `week`, `month` or `year`
DELETE	`/limits/{period}`	Remove a limit

```
PUT /v1/limits/week
{ "amount_paise": 50000, "is_active": true }

200 OK
{
  "period": "week",
  "amount_paise": 50000,
  "is_active": true,
  "period_start": "2026-10-12",
  "period_end": "2026-10-18",
  "spent_paise": 41000,
  "remaining_paise": 9000,
  "percent": 82,
  "status": "warning"
}
```

`status` is `ok` below 80%, `warning` from 80% to 99%, and `exceeded` from 100%. `GET /limits` returns the same object for each of the three periods that has a limit.

Alert behaviour

- Alerts are returned inside expense responses (`limit_status`) and in `GET /limits`, so every client shows the same message. No separate alert endpoint is needed in v1.

- Each level (80 and 100) fires once per limit per period. Lowering a limit mid-period can fire a new alert if the new percentage crosses a level not yet recorded.
- Deleting or editing an expense lowers the percentage but never removes an alert already shown.
- Phase 2 adds push notifications for the same two levels using the stored device tokens.

5. Reports and dashboard

Both endpoints are read-only and use the user's time zone and week start to work out the period.

Method	Path	Purpose
GET	`/reports/weekly?date=2026-10-13`	Weekly report for the week containing that date (default: this week)
GET	`/dashboard?range=week|month|year&date=2026-10-13`	Data for the three dashboard visuals

Weekly report

```
GET /v1/reports/weekly?date=2026-10-13

200 OK
{
  "period": { "start": "2026-10-12", "end": "2026-10-18" },
  "total_paise": 41000,
  "entries": 23,
  "vs_last_week": { "last_total_paise": 36000, "change_paise": 5000,
"change_percent": 13.9 },
  "top_expenses": [
    { "id": "...", "item_name": "Groceries", "amount_paise": 15000,
"spent_on": "2026-10-14" }
  ],
  "top_category": { "name": "Food", "total_paise": 18500 },
  "need_want": { "need_paise": 29000, "want_paise": 12000 },
  "top_items": [ { "item": "tea", "times": 9, "total_paise": 13500 } ],
  "daily": [ { "date": "2026-10-12", "total_paise": 5500 } ],
  "summary_text": "You spent ₹410 this week, ₹50 more than last week. ₹120
was on wants."
}
```

`top_expenses` holds up to 3 entries. `daily` always has one entry per day of the week, with 0 for days with no spending. `summary_text` is built in the user's language so web and mobile show the same sentence.

Dashboard

```
GET /v1/dashboard?range=month&date=2026-10-13

200 OK
{
  "period": { "start": "2026-10-01", "end": "2026-10-31" },
  "total_paise": 182000,
  "by_category": [
    { "category_id": "...", "name": "Food", "total_paise": 62000 },
    { "category_id": "...", "name": "Travel", "total_paise": 41000 }
  ],
  "daily": [ { "date": "2026-10-01", "total_paise": 4500 } ],
  "limits": [
    { "period": "month", "limit_paise": 300000, "spent_paise": 182000,
"percent": 61, "status": "ok" }
  ]
}
```

For `range=year`, `daily` is replaced by monthly totals (`monthly`), because 365 bars are not readable. The three parts map to the three dashboard visuals: category chart, daily (or monthly) bars, and limit progress.

6. Settings, sync, export, errors and endpoint summary

Settings and account

```
GET /v1/me
PATCH /v1/me
{ "language": "hi", "week_start": 1, "reminder_enabled": true,
"reminder_time": "21:00" }
```

`POST /devices` registers a push token for reminders. `DELETE /me` deletes the account and all data (the app asks for a confirmation first). `GET /exports/expenses.csv?from=...&to=...` downloads a CSV.

Sync (for offline use and guest data merge)

```
POST /v1/sync/push
{
  "expenses": [ { "id": "...", "item_name": "Auto", "amount_paise": 1000,
                  "spent_on": "2026-10-13", "tag": "need",
                  "updated_at": "2026-10-13T09:12:44Z", "deleted": false } ],
  "quick_add": [ ... ],
  "limits": [ ... ]
}

200 OK
{ "accepted": 42, "rejected": [ { "id": "...", "reason": "validation_error" }
] }

GET /v1/sync/changes?since=2026-10-13T09:00:00Z
200 OK
{ "expenses": [...], "categories": [...], "quick_add": [...], "limits":
[...],
  "server_time": "2026-10-13T09:30:00Z" }
```

The server accepts a pushed row only if its `updated_at` is newer than the stored one (latest write wins). The client stores `server_time` and sends it as `since` on its next call. A push can hold up to 500 rows.

Error codes

HTTP	`code`	When
400	`validation_error`	A field is missing or invalid (amount ⩽ 0, bad date, name too long)
401	`unauthorized`	Missing or expired access token
401	`invalid_otp`	Wrong or expired OTP
403	`forbidden`	The resource belongs to another user
404	`not_found`	Unknown ID
409	`conflict`	Limit or chip already exists in a different form
429	`rate_limited`	Too many requests
500	`server_error`	Unexpected failure; the client retries with the same ID

Endpoint summary

Area	Endpoints
Auth	`POST /auth/otp/request`, `/auth/otp/verify`, `/auth/google`, `/auth/refresh`, `/auth/logout`
Expenses	`PUT|GET|DELETE /expenses/{id}`, `GET /expenses`, `POST /expenses/{id}/restore`
Quick-add	`GET /quick-add`, `PUT|DELETE /quick-add/{id}`
Categories	`GET /categories`, `POST /categories`, `GET /categories/suggest`
Limits	`GET /limits`, `PUT|DELETE /limits/{period}`
Reports	`GET /reports/weekly`, `GET /dashboard`
Account	`GET|PATCH|DELETE /me`, `POST /devices`, `GET /exports/expenses.csv`
Sync	`POST /sync/push`, `GET /sync/changes`

Build notes. Write the API as an OpenAPI 3 file first; it can generate typed clients for React, React Native or Flutter and a test mock server. Total: 28 endpoints, all small enough to build and test in the web phase before any mobile work starts.